

PRACTICAL NO: 3

AIM:3. Finding Relationships Among Variables:

- Use a dataset with multiple variables and perform correlation analysis to determine the strength and direction of relationships between pairs of variables.
- Apply regression analysis to identify the relationship between an independent variable (e.g., advertising and venue).

The image displays three sequential screenshots of a Google Colab notebook titled "T110_TYCS_PRA_3_APPLIDE BUSSINESS AND ANALYTICS".

First Screenshot: Shows the initial code cells. The first cell installs and imports necessary libraries (pandas, numpy, matplotlib, seaborn, sklearn). The second cell uploads a file named "Advertising.csv" from Google Drive. The third cell reads the CSV file into a DataFrame named 'df'.

Second Screenshot: Shows the output of the first code cell. The DataFrame 'df' is displayed using the `df.head()` command, showing the first five rows of data. The columns are: Unnamed: 0, TV, Radio, Newspaper, Sales. The data values are as follows:

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

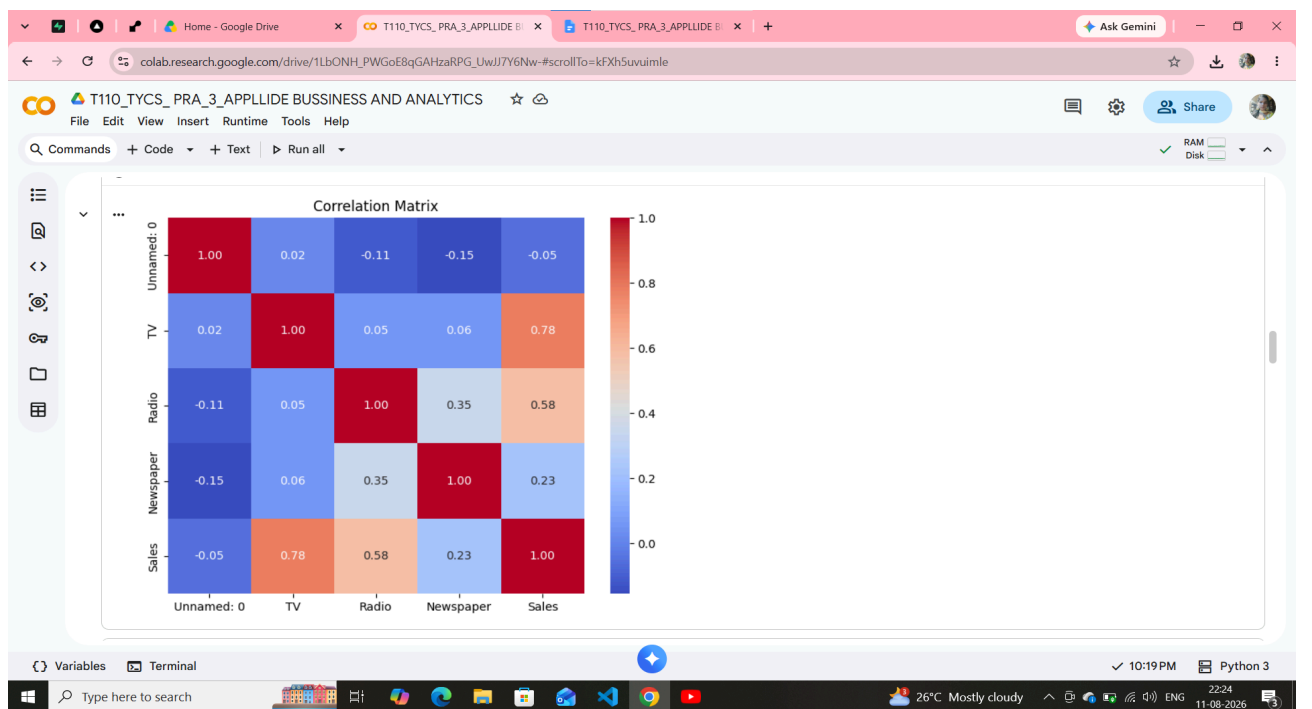
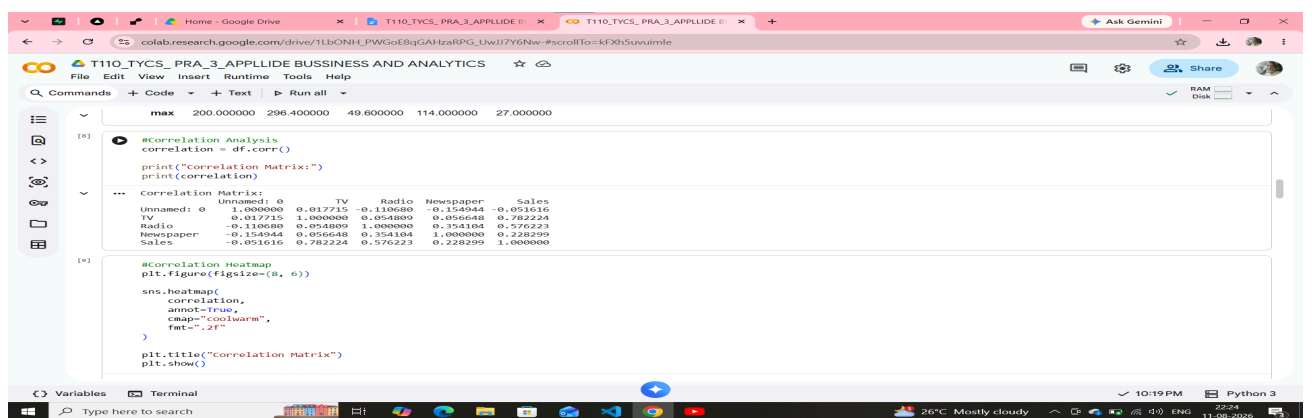
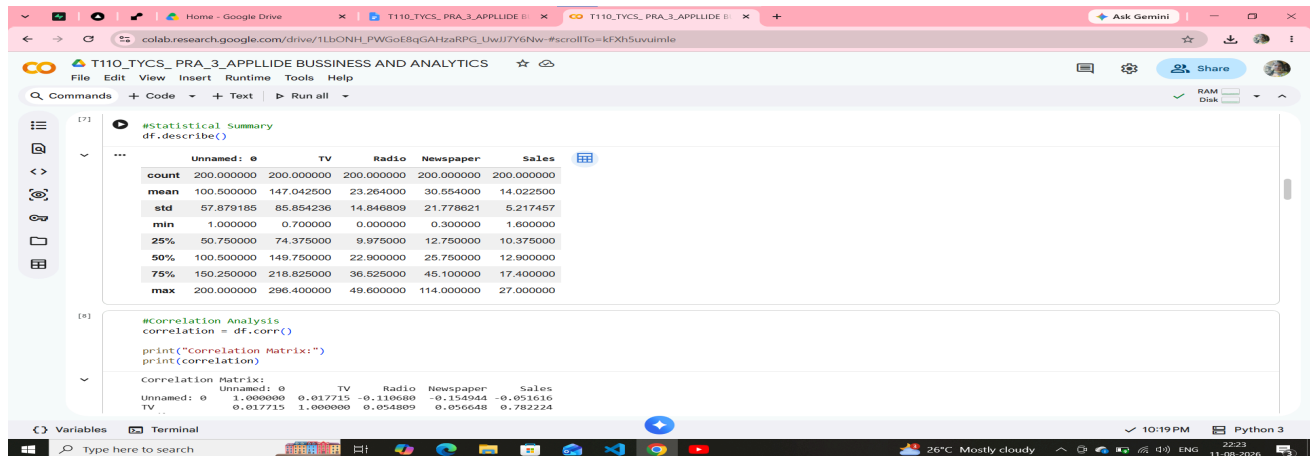
Below the table, the output of the `df.info()` command is shown, indicating the dataset shape is (200, 5).

Third Screenshot: Shows the output of the `df.info()` command, providing detailed information about the dataset, including the number of columns, data types, and memory usage. The output is as follows:

```
Dataset Shape:
(200, 5)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   Unnamed: 0    200 non-null   int64  
 1   TV            200 non-null   float64 
 2   Radio         200 non-null   float64 
 3   Newspaper     200 non-null   float64 
 4   Sales         200 non-null   float64 
dtypes: float64(4), int64(1)
memory usage: 7.9 KB
```

Below this, the output of the `df.isnull().sum()` command is shown, indicating no missing values in the dataset.



The image displays a Jupyter Notebook environment with three sequential code blocks and their outputs.

Block 1: TV Advertising vs Sales

```
[10]: #TV Advertising vs Sales
plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="TV",
    y="Sales"
)

plt.title("TV Advertising vs Sales")
plt.xlabel("TV Advertising")
plt.ylabel("Sales")

plt.show()
```

Block 2: #X and Y Define Karo

```
1: X = df[["TV"]]
y = df["Sales"]

print("Independent Variable: TV")
print("Dependent Variable: Sales")
```

Independent Variable: TV
Dependent Variable: Sales

Block 3: #Training aur Testing Data

```
1: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", len(X_train))
print("Testing data:", len(X_test))
```

Training data: 160
Testing data: 40

Block 4: #Linear Regression Model

```
[13]: model = LinearRegression()
model.fit(X_train, y_train)

print("Linear Regression Model Trained Successfully!")
```

Linear Regression Model Trained Successfully!

Block 5: #Regression Equation

```
[14]: print("Intercept:", model.intercept_)
print("Coefficient:", model.coef_[0])

print(
    "\nRegression Equation:"
)
print(
    "Sales =",
    model.intercept_,
    "+",
    model.coef_[0],
    "* TV"
)
```

Intercept: 7.119638430592953
Coefficient: 0.046529733705443346
Regression Equation:
Sales = 7.119638430592953 + 0.046529733705443346 * TV

```
[17] #R^2 Score
r2 = r2_score(y_test, y_pred)
print("R^2 Score:", r2)

R^2 Score: 0.6766954295627076

[18] #Mean Squared Error
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)

Mean Squared Error: 10.204654118800956

[19] #Final Regression Graph
plt.figure(figsize=(8, 5))

sns.regplot(
    data=df,
    x="TV",
    y="Sales"
)

plt.title("Linear Regression: TV Advertising vs Sales")
plt.xlabel("TV Advertising")
plt.ylabel("Sales")

plt.show()
```

